

Compte rendu de TP

Initiation au langage PL/pgSQL

Étudiant Délio Duchesne-Brajon
Bloc Bloc 2 — Développement
SGBD PostgreSQL — pgAdmin

SOMMAIRE

Objectif du TP	2
Exercice 1 — calculer_longueur_max()	2
Exercice 2 — nb_occurrences()	2
1. Version LOOP / EXIT — première tentative	2
2. Version LOOP / EXIT — corrigée.....	3
3. Version WHILE	3
Exercice 3 — getNbJoursParMois()	4
Exercice 4 — dateSqlToDatefr().....	5
Exercice 5 — getNomJour()	5
Exercice 6 — Contexte HHB-Direct.....	6
1. nb_clients_debiteurs()	6
2. nb_clients_ville()	7
Exercice 7 — inserer_client().....	8
Conclusion	9

Objectif du TP

Initiation au langage PL/pgSQL : écriture de fonctions manipulant des chaînes de caractères, des dates, puis interrogeant une base de données. Les fonctions ont été écrites et testées sous PostgreSQL.

Exercice 1 — calculer_longueur_max()

Fonction qui compare la longueur de deux chaînes de caractères et retourne la plus grande des deux.

```
hhbdirect=# CREATE OR REPLACE FUNCTION calculer_longueur_max(chaine1 varchar, chaine2 varchar) RETURNS integer AS $$
hhbdirect$# DECLARE
hhbdirect$#     long1 integer;
hhbdirect$#     long2 integer;
hhbdirect$# BEGIN
hhbdirect$#     long1 := length(chaine1);
hhbdirect$#     long2 := length(chaine2);
hhbdirect$#     IF long1 > long2 THEN
hhbdirect$#         RETURN long1;
hhbdirect$#     ELSE
hhbdirect$#         RETURN long2;
hhbdirect$#     END IF;
hhbdirect$# END;
hhbdirect$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
hhbdirect=# SELECT calculer_longueur_max('bonjour', 'salut');
calculer_longueur_max
-----
7
(1 ligne)

hhbdirect=# |
```

Exercice 2 — nb_occurrences()

Fonction qui compte le nombre d'occurrences d'un caractère dans un intervalle d'une chaîne, avec contrôle de la validité de l'intervalle et utilisation de SUBSTR(). Trois versions ont été écrites.

1. Version LOOP / EXIT — première tentative

Erreur rencontrée : la fonction lenght() n'existe pas. Il s'agit d'une faute de frappe, le bon nom étant length().

```

SQL Shell (psql)
postgres# CREATE FUNCTION nb_occurrences_loop (caractere VARCHAR, chaine VARCHAR, debut integer, fin integer) RETURNS integer AS $$
postgres# DECLARE
postgres#     nb integer := 0;
postgres#     i integer :=1;
postgres#     sous_chaine VARCHAR;
postgres#     longueur integer;
postgres# BEGIN
postgres#     IF debut < 1 OR fin> lenght(chaine) OR debut>fin THEN
postgres#         RAISE EXCEPTION 'Intervalle invalide';
postgres#     END IF;
postgres#     sous_chaine := substr(chaine,debut, fin-debut +1);
postgres#     longueur := lenght (sous_chaine);
postgres#     LOOP
postgres#         EXIT WHEN i>longueur;
postgres#         IF substr(sous_chaine,i,1)= caractere THEN
postgres#             nb := nb+ 1;
postgres#         END IF;
postgres#         i := i+1;
postgres#     END LOOP;
postgres#     RETURN nb;
postgres# END;
postgres# $$ LANGUAGE plpgsql;
postgres# CREATE FUNCTION
postgres# SELECT nb_occurrences_loop('e', 'superieur', 2, 6);
ERREUR: la fonction lenght(character varying) n'existe pas
LIGNE 1 : debut < 1 OR fin> lenght(chaine) OR debut>fin
                                ^
ASTUCE : Aucune fonction ne correspond au nom donné et aux types d'arguments.
Vous devez ajouter des conversions explicites de type.
REQUÊTE : debut < 1 OR fin> lenght(chaine) OR debut>fin
CONTEXTE : fonction PL/pgSQL nb_occurrences_loop(character varying,character varying,integer,integer), ligne 8 à IF
postgres#

```

2. Version LOOP / EXIT — corrigée

Après correction du nom de la fonction, la boucle LOOP avec sortie par EXIT WHEN retourne le résultat attendu.

The screenshot shows a PostgreSQL client window with the following SQL query:

```

1 CREATE OR REPLACE FUNCTION nb_occurrences_loop1(
2     car CHAR(1),
3     chaine VARCHAR,
4     debut INTEGER,
5     fin INTEGER
6 )
7 RETURNS INTEGER AS $$
8 DECLARE
9     compteur INTEGER := 0;
10    i INTEGER;
11 BEGIN
12     IF debut < 1 OR fin > LENGTH(chaine) OR debut > fin THEN
13         RAISE EXCEPTION 'Intervalle invalide : début=%, fin=%, longueur de la chaîne=%',
14             debut, fin, LENGTH(chaine);
15     END IF;
16
17     i := debut;
18     LOOP
19         EXIT WHEN i > fin;
20         IF SUBSTR(chaine, i, 1) = car THEN
21             compteur := compteur + 1;
22         END IF;
23         i := i + 1;

```

The output shows the function call and its result:

nb_occurrences_loop1
integer
1

3. Version WHILE

Même traitement écrit avec une boucle WHILE, la condition d'arrêt étant portée par la boucle elle-même.

Query

Query History

8

```

DECLARE
compteur INTEGER := 0;
i INTEGER;
BEGIN
IF debut < 1 OR fin > LENGTH(chaine) OR debut > fin THEN
RAISE EXCEPTION 'Intervalle invalide : début=%, fin=%, longueur de la chaîne=%',
debut, fin, LENGTH(chaine);
END IF;

i := debut;
WHILE i <= fin LOOP
IF SUBSTR(chaine, i, 1) = caractere THEN
compteur := compteur + 1;
END IF;
i := i + 1;
END LOOP;

RETURN compteur;
END;
$$ LANGUAGE plpgsql;

SELECT nb_occurrences_while('a', 'anticonstitutionnellement', 1, 25);

```

car

↑ ↓

Data Output

Messages

Notifications

nb_occurrences_while

integer

1

1

Exercice 3 — getNbJoursParMois()

Fonction qui calcule le nombre de jours du mois correspondant à une date fournie en paramètre.

Query

Query History

1

```

CREATE OR REPLACE FUNCTION getNbJoursParMois(date_param DATE)
RETURNS INTEGER AS $$
DECLARE
dernier_jour DATE;
BEGIN
dernier_jour := (date_trunc('month', date_param) + INTERVAL '1 month' - INTERVAL '1 day')::DATE;
RETURN EXTRACT(DAY FROM dernier_jour);
END;
$$ LANGUAGE plpgsql;
SELECT getNbJoursParMois('2026-03-27');

```

Data Output

Messages

Notifications

getnbjoursparmois

integer

1

31

Exercice 4 — dateSqlToDatefr()

Fonction qui convertit une date au format SQL (AAAA-MM-JJ) en date au format français (JJ/MM/AA).

The screenshot shows a SQL IDE interface with a 'Query' tab and a 'Query History' tab. The SQL code is as follows:

```
1  
2 CREATE OR REPLACE FUNCTION dateSqlToDatefr(date_param DATE)  
3 RETURNS VARCHAR AS $$  
4 BEGIN  
5     RETURN TO_CHAR(date_param, 'DD/MM/YY');  
6 END;  
7 $$ LANGUAGE plpgsql;  
8 SELECT dateSqlToDatefr('2026-03-27');  
9 |
```

Below the query editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with the following data:

	datesqltodatefr character varying
1	27/03/26

Exercice 5 — getNomJour()

Fonction qui retourne le nom du jour de la semaine correspondant à une date fournie, à l'aide de la fonction EXTRACT().

The screenshot shows a PostgreSQL query editor with a toolbar at the top. The 'Query' tab is active, displaying a SQL script. The script defines a function `getNomJour` that takes a date parameter and returns the day of the week as a string. The function uses a `CASE` statement to map the day of the week (0 for Sunday to 6 for Saturday) to its corresponding name in French. The function is created using `CREATE OR REPLACE FUNCTION` and `LANGUAGE plpgsql`. The script concludes with a `SELECT` statement that calls the function with the date '2026-09-18'.

```
1 CREATE OR REPLACE FUNCTION getNomJour(date_param DATE)
2 RETURNS VARCHAR AS $$
3 DECLARE
4     jour_semaine INTEGER;
5     nom_jour VARCHAR(15);
6 BEGIN
7     jour_semaine := EXTRACT(DOW FROM date_param);
8
9     CASE jour_semaine
10     WHEN 0 THEN nom_jour := 'Dimanche';
11     WHEN 1 THEN nom_jour := 'Lundi';
12     WHEN 2 THEN nom_jour := 'Mardi';
13     WHEN 3 THEN nom_jour := 'Mercredi';
14     WHEN 4 THEN nom_jour := 'Jeudi';
15     WHEN 5 THEN nom_jour := 'Vendredi';
16     WHEN 6 THEN nom_jour := 'Samedi';
17     END CASE;
18
19     RETURN nom_jour;
20 END;
21 $$ LANGUAGE plpgsql;
22 SELECT getNomJour('2026-09-18');
```

Below the query editor, the 'Data Output' tab is active, showing the result of the query. The result is a single row with the value 'Vendredi'.

	getnomjour character varying
1	Vendredi

Exercice 6 — Contexte HHB-Direct

Deux fonctions interrogeant la base bancaire HHB-Direct.

1. nb_clients_debiteurs()

Un client est considéré débiteur s'il possède au moins un compte au solde négatif (jointure client → posséder → compte).

```
1 CREATE OR REPLACE FUNCTION nb_clients_debiteurs()
2 RETURNS INTEGER AS $$
3 DECLARE
4     nb INTEGER;
5 BEGIN
6     SELECT COUNT(DISTINCT p.num_client) INTO nb
7     FROM posseder p
8     JOIN compte c ON p.num_compte = c.num_compte
9                  AND p.id_type = c.id_type
10    WHERE c.solde < 0
11          AND (c.date_fermer IS NULL
12              OR date_part('year', c.date_fermer) >= date_part('year', CURRENT_DATE));
13    RETURN nb;
14 END;
15 $$ LANGUAGE plpgsql;
16
17 SELECT nb_clients_debiteurs();
```

Data Output Messages Notifications

nb_clients_debiteurs	
integer	
1	1

2. nb_clients_ville()

Compte les clients rattachés, pour l'année en cours, à une agence dont l'adresse contient la ville donnée (jointure affecter → agence), avec utilisation de `date_part()`.

Query

Query History

```

1 CREATE OR REPLACE FUNCTION nb_clients_ville(p_ville VARCHAR)
2 RETURNS INTEGER AS $$
3 DECLARE
4     nb INTEGER;
5 BEGIN
6     SELECT COUNT(DISTINCT a.num_client) INTO nb
7     FROM affecter a
8     JOIN agence ag ON a.num_agence = ag.num_agence
9     WHERE ag.adresse_agence ILIKE '%' || p_ville || '%'
10    AND date_part('year', a.date) = date_part('year', CURRENT_DATE);
11     RETURN nb;
12 END;
13 $$ LANGUAGE plpgsql;
14 SELECT nb_clients_ville('Manosque');
15

```

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	nb_clients_ville	integer
1		0

Exercice 7 — inserer_client()

Fonction qui insère un nouveau client dans la table client. Le numéro de client est calculé automatiquement (dernier numéro + 1) et la fonction retourne un message indiquant si l'enregistrement a réussi ou échoué.

```
Query Query History
4      p_adresse VARCHAR,
5      p_identifiant VARCHAR,
6      p_mdp VARCHAR
7  )
8  RETURNS VARCHAR AS $$
9  DECLARE
10     nouveau_num_client INTEGER;
11  BEGIN
12      SELECT COALESCE(MAX(num_client), 0) + 1 INTO nouveau_num_client FROM client;
13
14      INSERT INTO client(num_client, nom_client, prenom_client, adresse_client,
15                        identifiant_internet, mdp_internet)
16      VALUES (nouveau_num_client, p_nom, p_prenom, p_adresse, p_identifiant, p_mdp);
17
18      RETURN 'Client enregistré avec succès (n° ' || nouveau_num_client || ')';
19
20  EXCEPTION
21      WHEN OTHERS THEN
22          RETURN 'Erreur lors de l'enregistrement : ' || SQLERRM;
23  END;
24  $$ LANGUAGE plpgsql;
25
26  SELECT inserer_client('Duchesne-Brajon', 'Délío', '13 rue de Valserre, Gap', 'ddb05', 'delio');
```

Data Output Messages Notifications

	inserer_client character varying
1	Client enregistré avec succès (n° ...

Conclusion

Ce TD couvre les bases du PL/pgSQL : déclaration de variables, structures de contrôle, boucles LOOP et WHILE, gestion des exceptions et interrogation de la base depuis une fonction. L'exercice 2 montre qu'un même traitement admet plusieurs écritures, et l'exercice 7 que la gestion des erreurs fait partie intégrante de la fonction.